

Lab Report 5: GPIO Interfacing and Logic Control

Active High/Low, Pull-Up/Down, and Interrupt Handling

Students: Yossi Brim & Elad asbag

May 2026

1 Introduction

This laboratory focuses on General Purpose Input/Output (GPIO) operations within an Embedded Linux environment. The transition from software logic to physical hardware interaction is demonstrated through LED actuation, button input sensing, and the implementation of a 7-Segment digital counter. Key engineering concepts explored include active-high/low circuit topologies, pull-up/down resistor configurations, and efficient asynchronous event handling using interrupts.

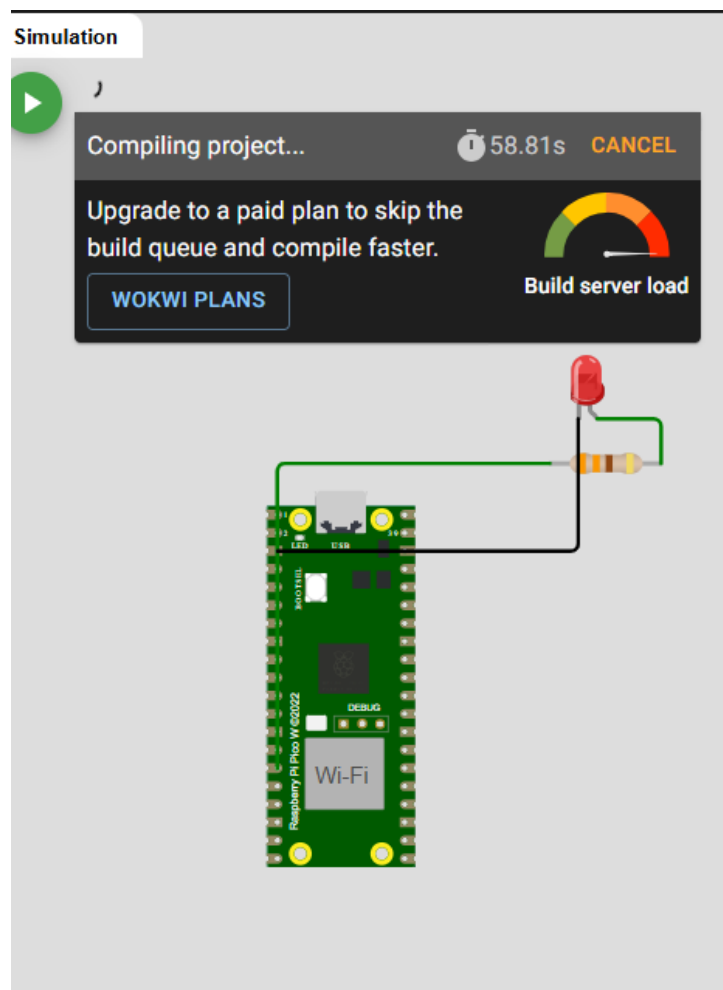
Methodology Note (Circuit Documentation): To ensure maximum clarity and a professional presentation, the circuit diagrams in this report were generated using the Wokwi simulation environment. This approach was explicitly chosen over photographing physical breadboards, as it provides a cleaner, standardized, and highly readable schematic layout for analyzing the hardware connections.

2 Task 1: Frequency-Controlled LED (Active High-/Low)

2.1 Objective & Expectations

The objective was to toggle a LED on GPIO 11 at a constant frequency of 1Hz. **Expectation:** We expected the LED to blink regardless of the hardware topology (Active High or Active Low), provided the software toggles the logical state. In Active High, the LED should light up on `GPIO.HIGH`. In Active Low, it should light up on `GPIO.LOW`.

2.2 Circuit Documentation



2.3 Python Implementation

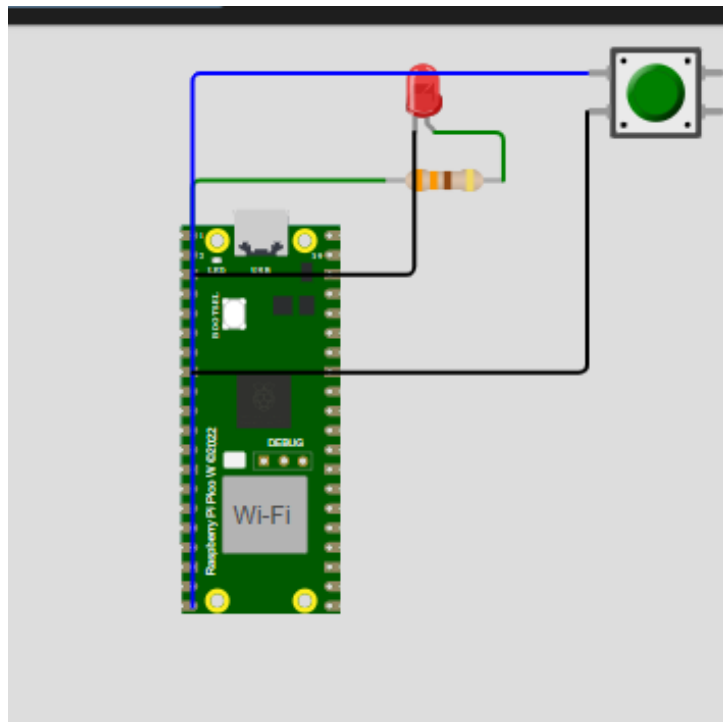
```
1 import RPi.GPIO as GPIO
2 import time
3
4 GPIO.setmode(GPIO.BCM)
5 LED_PIN = 11
6 GPIO.setup(LED_PIN, GPIO.OUT)
7
8 try:
9     while True:
10         GPIO.output(LED_PIN, GPIO.HIGH)
11         time.sleep(0.5)
12         GPIO.output(LED_PIN, GPIO.LOW)
13         time.sleep(0.5)
14 except KeyboardInterrupt:
15     GPIO.cleanup()
```

3 Task 2: Interrupt-Driven Input and Frequency Scaling

3.1 Objective & Expectations

The goal was to utilize a push-button on GPIO 15 to dynamically double the blink frequency of the LED. **Expectation:** Using the internal Pull-Up resistor (PUD_UP), the pin should read HIGH by default. A press (Active Low) will trigger a falling edge. We utilized **Interrupts** (edge detection) rather than polling to ensure instantaneous response without blocking the main execution loop.

3.2 Circuit Documentation



3.3 Python Implementation

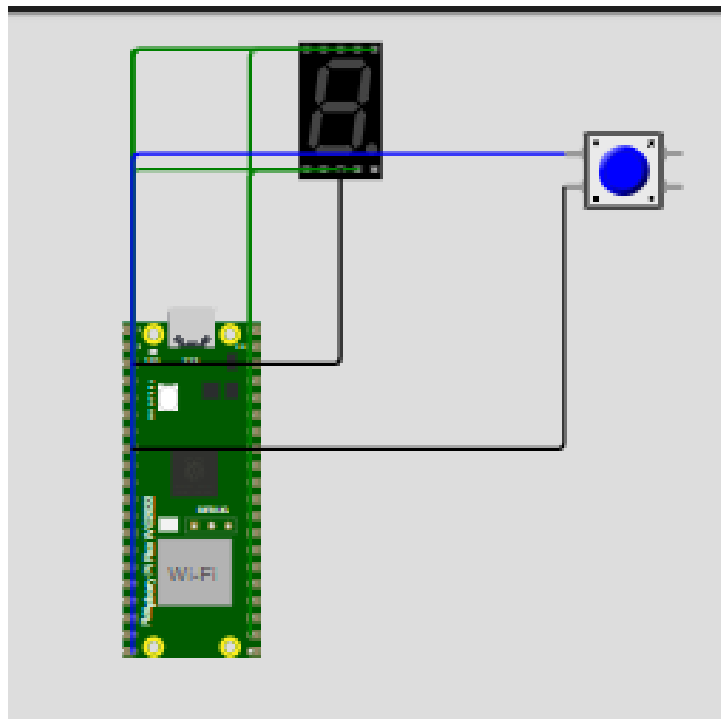
```
1 # Key snippet from part_b.py
2 def button_pressed_callback(channel):
3     global blink_delay
4     blink_delay = blink_delay / 2.0 # Double frequency
5     if blink_delay < 0.05:
6         blink_delay = 0.5 # Reset to 1Hz
7
8 GPIO.setup(BUTTON_PIN, GPIO.IN, pull_up_down=GPIO.PUD_UP)
9 GPIO.add_event_detect(BUTTON_PIN, GPIO.FALLING, callback=
    button_pressed_callback, bouncetime=300)
```

4 Task 3: 7-Segment Digital Counter with Directional Control

4.1 Objective & Expectations

Implementation of a 0-9 counter on a 7-segment display, where a button toggle reverses the counting direction (Up/Down). **Expectation:** We mapped the segments A-G to specific GPIO pins. We expected the state machine to maintain the current count while the interrupt-driven button toggles a boolean direction flag.

4.2 Circuit Documentation



5 Conclusion and Engineering Insights

This laboratory provided a comprehensive exploration of hardware-software integration using the Raspberry Pi GPIO interface. The transition from software-defined logic to physical circuit actuation highlighted several critical embedded systems concepts:

- **Hardware Abstraction:** By testing both Active High and Active Low topologies, we demonstrated that hardware polarity can be seamlessly abstracted and managed via software logic, allowing for greater flexibility in component selection and printed circuit board (PCB) design.
- **Resource Optimization:** Utilizing the SoC's internal pull-up/pull-down resistors significantly reduced the need for external discrete components. This approach not only simplifies the physical schematic but also minimizes potential hardware failure points.

- **Event-Driven Architecture vs. Polling:** A major engineering takeaway was the implementation of hardware interrupts. Replacing blocking `while` loops (polling) with edge-triggered callbacks (`GPIO.FALLING`) ensures that the CPU is not unnecessarily monopolized by input checking. This is a fundamental requirement for designing responsive, real-time embedded Linux systems.
- **State Machine Logic:** The 7-segment display counter successfully demonstrated how to map complex visual outputs to a software-defined dictionary. This effectively created a deterministic state machine that can be manipulated asynchronously by external hardware interrupts.

Ultimately, the lab bridged the gap between theoretical software execution and practical electronic engineering, proving that efficient, event-driven code is essential for reliable hardware manipulation.